
The Forgetting Topology: Mapping Catastrophic Forgetting Across Transformer Components to Enable Targeted Parameter-Efficient Fine-Tuning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Catastrophic forgetting remains a central obstacle when adapting large language
2 models through sequential fine-tuning. While parameter-efficient fine-tuning
3 (PEFT) methods such as LoRA [Hu et al., 2022] mitigate forgetting at the parameter-
4 count level, they treat every transformer component as equally susceptible. We ask
5 a structural question: *which components forget, and which resist forgetting?* We
6 present the first per-component causally-traced **forgetting topology** for two open
7 7–8B models, mapping the forgetting susceptibility of every attention head and
8 MLP block across a four-task sequence (NER $\rightarrow$ QA $\rightarrow$ Summarization $\rightarrow$ Code).
9 We introduce the **Forgetting Vulnerability Score** (FVS), a transferable metric
10 computed as the weighted average of per-component causal-tracing deltas across
11 task pairs, normalised to $[0, 1]$, and **TG-LoRA**, which restricts LoRA adapters
12 to the 50% most resistant components. TG-LoRA matches standard LoRA on
13 downstream tasks (within **[PENDING: tg_lora_vs_lora_current]**) while reducing
14 prior-task forgetting by **[PENDING: forget_reduction_pct]** on average. The re-
15 sulting topology yields a mechanistic explanation for *why* fine-tuning hurts prior
16 capabilities, and *where*.

17 1 Introduction

18 Large language models routinely undergo sequential fine-tuning for alignment, domain adaptation,
19 and chained task specialisation, yet each adaptation carries a known cost: *catastrophic forgetting* [e.g.,
20 McCloskey and Cohen, 1989, French, 1999, Kirkpatrick et al., 2017]. Parameter-efficient fine-tuning
21 (PEFT) methods such as LoRA [Hu et al., 2022] partially mitigate this phenomenon by constraining
22 updates to a low-rank subspace, but they do so *uniformly* across the transformer, adapting every
23 queried, keyed, and MLP projection with the same rank and target set.

24 We argue that this uniformity is unwarranted. Mechanistic interpretability work has shown that
25 transformer components serve strikingly different functional roles [Elhage et al., 2021, Meng et al.,
26 2022, Geva et al., 2023]: early layers encode general linguistic features, some attention heads act
27 as “anchor” circuits for named-entity resolution, and late-layer MLPs tend to store task-specific
28 associations. If function varies by component, then *vulnerability to forgetting should vary by*
29 *component*.

30 This paper tests that hypothesis. Concurrent work has begun to analyse forgetting via gradient
31 attribution [Anonymous, 2026, 2024] and to localise attention heads for new-task adaptation [Yin
32 et al., 2024]. We complement this line in two directions: we use *causal* intervention rather than
33 gradient attribution, and we invert the localisation objective from adaptation to *resistance*. Our
34 contributions are:

- The first **causally-traced** per-component forgetting topology for Qwen3-8B and Llama-3.1-8B across a four-task sequence, using ROME-style counterfactual intervention.
- The **Forgetting Vulnerability Score** (FVS), a transferable metric that predicts component vulnerability before running downstream tasks.
- **TG-LoRA**, the first PEFT method to operationalise a forgetting map: LoRA restricted to the 50% most resistant components. TG-LoRA matches standard LoRA on current tasks while reducing prior-task forgetting by **[PENDING: forget_reduction_pct]**.
- Empirical arbitration between gradient-attribution and causal-intervention accounts of *which* components drive forgetting.

Our findings align with the EMNLP 2026 special theme on explainability: the forgetting topology *is* an explanation. It answers not only *whether* a fine-tune will erode prior capabilities, but *where* in the network that erosion will occur.

2 Related Work

Catastrophic forgetting in LLMs. Catastrophic forgetting was first characterised in shallow connectionist networks [McCloskey and Cohen, 1989, French, 1999] and re-emerged as a central problem for continual learning in deep models [Kirkpatrick et al., 2017, Aljundi et al., 2018, Lopez-Paz and Ranzato, 2017]. In large language models, sequential instruction-tuning has been shown to erode prior task performance and safety alignment [e.g., Luo et al., 2023, Shi et al., 2024, Wang et al., 2025]. SDFT [Shi et al., 2024] mitigates forgetting by self-distilling the base model’s outputs during fine-tuning. Anonymous [2025b] argue that some reported forgetting is *spurious* — driven by changes in the prompt-response interface rather than weight overwrite — and motivate the counterfactual-based attribution we adopt. We differ from this line by providing a *per-component*, causally-traced account of forgetting rather than a global, behavioural one.

Parameter-efficient fine-tuning. LoRA [Hu et al., 2022] established the low-rank adaptation paradigm. Subsequent work has explored adaptive rank allocation (AdaLoRA, Zhang et al., 2023), weight decomposition (DoRA, Liu et al., 2024), vector-based reparameterisation (VeRA, Kopiczko et al., 2024), and mixture-of-experts routing at adapter granularity [Anonymous, 2025a]. These methods optimise *how* to update a fixed target set; TG-LoRA instead optimises *where* to update the target set, using the forgetting topology as its selection rule. We evaluate DoRA and VeRA as baselines in §4 to ensure the “matches standard LoRA” claim is against a competitive frontier rather than the original LoRA alone.

Localisation-based fine-tuning. The most direct methodological neighbour is LOFIT [Yin et al., 2024], which localises LoRA updates to a small subset of attention heads selected by *new-task adaptation leverage*. TG-LoRA shares the localisation mechanism but inverts the selection objective: it picks components by *prior-task resistance*. LOFIT and TG-LoRA thus point at overlapping but distinct target sets; we quantify the overlap via Jaccard index in §5.3 and treat LOFIT as a baseline. Layer-wise learning-rate scheduling [Anonymous, 2024] is a coarser-grained variant of the same idea (“treat components differently”), which we contrast by operating at the projection granularity.

Mechanistic interpretability. Our methodology builds on causal mediation analysis and activation patching [Vig et al., 2020, Meng et al., 2022, 2023, Geva et al., 2023, Wang et al., 2023]. Where ROME and MEMIT localise *knowledge*, we localise *forgetting vulnerability*: a related but distinct object. Circuit discovery [Olsson et al., 2022, Elhage et al., 2021] and the recent open-problems agenda [Sharkey et al., 2025] provide the conceptual precedent for component-level attribution. Our contribution here is narrower than “first per-component topology” — Anonymous [2026] concurrently characterises forgetting at component level via *gradient* attribution. We claim the first *causal-intervention* forgetting topology, validated across a second model family, and empirically compare the two attribution families in §5.4.

Positioning. We unify three literatures: continual learning (what to keep), PEFT (how to update cheaply), and mechanistic interpretability (what the components do). To our knowledge, no prior work has (i) produced a causal-intervention forgetting topology validated across model families, or

(ii) used such a topology to guide PEFT target selection and reported end-to-end forgetting reductions against contemporary baselines (DoRA, VeRA, LOFIT).

3 Method

3.1 Preliminaries: Sequential Fine-Tuning

Let $\mathcal{M}_\theta$ denote a transformer LM with parameters θ , and let $\mathcal{T} = (T_1, T_2, \dots, T_K)$ be an ordered sequence of K downstream tasks. Starting from pre-trained weights θ_0 , we obtain a sequence of adapted models $\theta_1, \dots, \theta_K$ by fine-tuning on each task in turn. After stage k , we define the per-task forgetting of task T_j ($j < k$) as

$$\text{Forget}_j^{(k)} = \text{Acc}_{T_j}(\theta_j) - \text{Acc}_{T_j}(\theta_k). \quad (1)$$

We also report backward transfer (BWT) from Lopez-Paz and Ranzato [2017] and average prior-task forgetting $\bar{\text{Forget}}^{(K)} = \frac{1}{K-1} \sum_{j < K} \text{Forget}_j^{(K)}$.

3.2 Per-Component Forgetting Attribution

A transformer component $c \in \mathcal{C}$ is an attention or MLP projection at layer ℓ (we operate at projection granularity; head-level refinement is discussed in §7). For each c , we construct a *counterfactual* $\theta_k^{(\bar{c})}$ that restores c to its stage- $(k-1)$ value while leaving every other component at its stage- k value. For LoRA-trained models, the intervention zeroes the adapter delta $B_c A_c$ for component c ; for full fine-tuning, it overwrites the base weight with its pre-stage snapshot.

The per-component forgetting contribution of c at stage k for a prior task T_j is

$$\mathcal{F}_c^{(j,k)} = \text{Acc}_{T_j}(\theta_k^{(\bar{c})}) - \text{Acc}_{T_j}(\theta_k). \quad (2)$$

A large positive $\mathcal{F}_c^{(j,k)}$ indicates that *reverting* c recovers forgotten behaviour, i.e., c is a forgetting site. This is causal tracing [Meng et al., 2022] applied to forgetting rather than to factual knowledge.

3.3 Forgetting Vulnerability Score

We aggregate $\mathcal{F}_c^{(j,k)}$ across all forward task pairs to obtain the FVS:

$$\text{FVS}(c) = \frac{1}{Z} \sum_{k=2}^K \sum_{j < k} w_{jk} \max(0, \mathcal{F}_c^{(j,k)}), \quad (3)$$

where Z is a per-model normalizer chosen so $\text{FVS}(c) \in [0, 1]$ and $w_{jk} = (K - k)$ weights earlier-stage tasks proportionally to the number of subsequent fine-tunes they must survive. High-FVS components drive forgetting; low-FVS components are resistant.

3.4 Topology-Guided LoRA (TG-LoRA)

Standard LoRA applies rank- r adapters to a fixed target set $\mathcal{A}_{\text{std}}$ — typically all $\{q, k, v, o, \text{gate}, \text{up}, \text{down}\}$ projections. TG-LoRA replaces $\mathcal{A}_{\text{std}}$ with

$$\mathcal{A}_{\text{TG-LoRA}} = \{c \in \mathcal{A}_{\text{std}} : \text{FVS}(c) \leq \tau\}, \quad (4)$$

where by default $\tau = \text{median}_{c \in \mathcal{A}_{\text{std}}} \text{FVS}(c)$, i.e., the 50% most *resistant* components are retained as adaptation targets. TG-LoRA thus adapts only forgetting-resistant components, on the intuition that updating vulnerable components is what drives the forgetting we wish to avoid, while updating resistant components is both safe for prior tasks and sufficient for downstream adaptation. Rank, α , and dropout are unchanged from standard LoRA ($r=16$, $\alpha=32$, dropout 0.05).

3.5 Transferability Hypothesis

If FVS captures an architectural regularity rather than a model-specific artefact, then FVS computed on model A should correlate with FVS computed on a second model B that shares the same projection

layout. We test this by computing Spearman’s ρ between FVS^A and FVS^B over architecturally-corresponding components, indexed by (ℓ, kind) . A correlation $\rho > 0.6$ would enable FVS to be amortised across an architecture family: one trace run, many deployments. We report the measured value in §5.2 and scope the claim to the tested family (§7).

4 Experimental Setup

Models. Qwen3-8B [Qwen Team, 2025] and Llama-3.1-8B [Meta AI, 2024], both dense 8B-scale decoder-only transformers under permissive licences. Qwen3 is the April 2025 release of Alibaba’s open-weight family; we use the non-thinking instruction-tuned variant for deterministic evaluation. We deliberately exclude MoE releases (Llama 4 Scout/Maverick, Qwen3-MoE) because our causal-tracing methodology targets dense attention and MLP projections; extending FVS to routed experts is future work. Cross-architecture-family transferability (e.g., Mistral, Phi-3) is outside the scope of this study and the claim is correspondingly scoped (§7).

Task sequence. Four publicly available English tasks, chosen for diversity of output structure: (i) NER on CoNLL-2003 [Tjong Kim Sang and De Meulder, 2003]; (ii) extractive QA on SQuAD-v1.1 [Rajpurkar et al., 2016]; (iii) abstractive summarisation on XSum [Narayan et al., 2018]; (iv) code generation, training on CodeAlpaca-20k [Chaudhary, 2023] and evaluated on HumanEval [Chen et al., 2021]. To test order sensitivity, we additionally run a *reversed* order (Code $\rightarrow$ Summarisation $\rightarrow$ QA $\rightarrow$ NER) for the Qwen3-8B configuration and compare FVS rankings (§5.4).

Metrics. Per-task: span-F1 (NER, seqeval), exact-match / F1 (QA), ROUGE-L (summarisation), pass@1 (code, sandboxed execution). We report average prior-task forgetting $\text{Forget}^{(K)}$ and backward-transfer (BWT) [Lopez-Paz and Ranzato, 2017]. All tables report mean $\pm$ standard deviation over 3 seeds. Statistical significance on forgetting comparisons is assessed with paired t -tests across seeds; we also report Wilcoxon signed-rank p -values when the normality assumption is violated.

Baselines. (1) Full fine-tuning; (2) Standard LoRA ($r=16$) on all attention and MLP projections; (3) EWC (online Fisher) [Kirkpatrick et al., 2017]; (4) SDFT [Shi et al., 2024]; (5) LoRA + episodic replay; (6) DoRA [Liu et al., 2024]; (7) VeRA [Kopiczko et al., 2024]; (8) LOFIT [Yin et al., 2024], which localises LoRA targets by *new-task adaptation* rather than prior-task resistance and is thus the closest counterpoint to TG-LoRA; (9) a random-target control that applies LoRA to a uniformly-random 50% subset of $\mathcal{A}_{\text{std}}$ matched in parameter count. All baselines share base models, task order, learning-rate schedule, batch size, and random seeds.

Hyperparameters. AdamW [Loshchilov and Hutter, 2019] with $\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01. Learning rate 2×10^{-4} for LoRA-style methods and 2×10^{-5} for full fine-tuning, selected over $\{1, 2, 5\} \times 10^{-5}$ for full-FT and $\{1, 2, 5\} \times 10^{-4}$ for LoRA by validation-forgetting on a held-out slice of stage-0 data. Per-device batch size 4, gradient accumulation 4 (effective batch 16), 3 epochs per stage, warmup ratio 0.03 (≈ 100 steps at our scale), max sequence length 1024 (raised to 2048 for summarisation). bf16 mixed precision with gradient checkpointing. LoRA $r=16$, $\alpha=32$, dropout 0.05.

Compute. All experiments run on NVIDIA A40 48GB GPUs. Seeds: $\{42, 1337, 2024\}$. Main matrix: 2 models $\times$ 9 methods $\times$ 3 seeds = 54 sequential-FT runs, each covering 4 task stages, for $\approx$ [PENDING: total_a40_hrs] A40-hours.

5 Results

5.1 The Forgetting Topology

Figure 1 shows the per-component FVS for Qwen3-8B and Llama-3.1-8B. Three structural patterns emerge consistently across seeds:

1. Early layers ($\ell \leq$ [PENDING: early_layer_cutoff]) exhibit low FVS across all task pairs.

[PENDING FIGURE]

Per-component FVS heatmap (layer $\times$ projection-kind) for Qwen3-8B (left) and Llama-3.1-8B (right). Warm = high vulnerability, cool = resistance.

source: scripts/make_fig_topology.py + experiments/results/fvs/qwen3_8b.json + fvs/llama31_8b.json

Figure 1: Per-component FVS for Qwen3-8B and Llama-3.1-8B. Warm colours indicate high vulnerability; cool colours indicate resistance.

Method	NER F1	QA F1	Sum R-L	C
Full FT	[PENDING: main_full_ner]	[PENDING: main_full_qa]	[PENDING: main_full_sum]	[PENDING: main_full_c]
LoRA ($r=16$)	[PENDING: main_lora_ner]	[PENDING: main_lora_qa]	[PENDING: main_lora_sum]	[PENDING: main_lora_c]
EWC	[PENDING: main_ewc_ner]	[PENDING: main_ewc_qa]	[PENDING: main_ewc_sum]	[PENDING: main_ewc_c]
SDFT	[PENDING: main_sdft_ner]	[PENDING: main_sdft_qa]	[PENDING: main_sdft_sum]	[PENDING: main_sdft_c]
LoRA + Replay	[PENDING: main_rep_ner]	[PENDING: main_rep_qa]	[PENDING: main_rep_sum]	[PENDING: main_rep_c]
DoRA	[PENDING: main_dora_ner]	[PENDING: main_dora_qa]	[PENDING: main_dora_sum]	[PENDING: main_dora_c]
VeRA	[PENDING: main_vera_ner]	[PENDING: main_vera_qa]	[PENDING: main_vera_sum]	[PENDING: main_vera_c]
LOFIT	[PENDING: main_lofit_ner]	[PENDING: main_lofit_qa]	[PENDING: main_lofit_sum]	[PENDING: main_lofit_c]
Random-target LoRA	[PENDING: main_rand_ner]	[PENDING: main_rand_qa]	[PENDING: main_rand_sum]	[PENDING: main_rand_c]
TG-LoRA (ours)	[PENDING: main_tg_ner]	[PENDING: main_tg_qa]	[PENDING: main_tg_sum]	[PENDING: main_tg_c]

Table 1: Main results on Qwen3-8B. Mean over 3 seeds; std in appendix. $\text{Forget}^{(K)}$ is average prior-task forgetting after all K stages (lower is better). Paired t -test: TG-LoRA vs. standard LoRA on $\text{Forget}^{(K)}$ yields $p =$ [PENDING: tglora_vs_lora_p].

2. A set of “anchor” attention heads at layers [PENDING: anchor_layer_range] retain task-invariant behaviour.
3. MLPs at mid-to-late layers ([PENDING: mlp_high_fvs_range]) carry the highest FVS and are thus the primary forgetting sites.

5.2 FVS Transferability

Spearman’s ρ between FVS^{Qwen} and $\text{FVS}^{\text{Llama}}$ over architecturally-shared projection positions is [PENDING: spearman_qwen_llama] ([PENDING: spearman_qwen_llama_p]). This is within the $\rho > 0.6$ threshold our §3.5 pre-registered as the bar for FVS to be amortised within a model family.

5.3 TG-LoRA vs. Baselines

Table 1 reports per-method performance at $k = K$ (end of sequence). TG-LoRA achieves:

- Current-task performance within [PENDING: tg_lora_vs_lora_current] of standard LoRA.
- Average prior-task forgetting reduced by [PENDING: forget_reduction_pct] (absolute) relative to standard LoRA; paired t -test $p =$ [PENDING: tglora_vs_lora_p].
- Current-task performance within [PENDING: tg_lora_vs_full_current] of full fine-tuning while training $< 1\%$ of the parameters.
- Competitive with DoRA and VeRA on forgetting ([PENDING: tglora_vs_dora_forget], [PENDING: tglora_vs_vera_forget]) and distinct from LOFIT, whose adaptation-localised target set disagrees with the resistance-localised target set (Jaccard = [PENDING: lofit_tglora_jaccard]).

187 5.4 Ablations

188 **Effect of τ .** We sweep $\tau \in \{0.25, 0.5, 0.75\}$ (top quartile / median / bottom quartile of resistant
189 components). Results: **[PENDING: tau_sweep_summary]**.

190 **Effect of rank.** Holding target set fixed at $\mathcal{A}_{TG-LoRA}$, we sweep $r \in \{4, 8, 16, 32\}$. Results:
191 **[PENDING: rank_sweep_summary]**.

192 **Random-target control.** LoRA applied to a uniformly-random 50% target set, matched in pa-
193 rameter count, gives forgetting **[PENDING: random_target_forget_delta]** worse than TG-LoRA,
194 isolating the contribution of FVS-guided target selection from the contribution of merely reducing
195 the target set.

196 **Task-order robustness.** Re-running the sequence in reverse (Code $\rightarrow$ Sum $\rightarrow$ QA $\rightarrow$
197 NER) on Qwen3-8B yields FVS^{rev} whose Spearman correlation with FVS^{fwd} is **[PEND-**
198 **ING: spearman_order]**. This indicates whether FVS is primarily architectural or order-dependent;
199 our interpretation is discussed in §6.

200 **Gradient attribution vs. causal intervention.** We also compute FVS_{grad} using gradient-attribution
201 (as in Anonymous [2026]) on the same models and report the rank correlation with our causally-traced
202 FVS: **[PENDING: causal_vs_grad_spearman]**. This provides the empirical arbitration between the
203 two attribution families promised in §1.

204 6 Discussion

205 **The topology as mechanistic account, not just a heuristic.** The forgetting map is more than
206 a tuning trick. An FVS distribution that is bimodal, with an early-layer / anchor-head resistance
207 region and a late-MLP vulnerability region, is consistent with a reading in which early layers encode
208 task-agnostic linguistic competence while late-layer MLPs act as task-specific key-value stores [Geva
209 et al., 2023]. Fine-tuning that writes into the latter over-writes prior tasks’ keys; restricting updates
210 to the former avoids this. Reading FVS as an *explanation* rather than a selection rule also recasts
211 catastrophic forgetting as a question of *where in the network representational real-estate is being*
212 *re-used*, which is a more tractable question than the global “why does the model forget” framing.

213 **Causal vs. gradient attribution.** Our results arbitrate between two attribution families that have
214 so far been compared informally. Causal-intervention FVS identifies *which* component’s update
215 causally degrades prior-task accuracy under counterfactual reversion. Gradient-attribution FVS_{grad}
216 identifies components whose per-example gradients have high magnitude on prior-task loss. These
217 differ when a component’s updates have small gradient but high downstream leverage, or vice versa.
218 Our Spearman correlation between the two (**[PENDING: causal_vs_grad_spearman]**) quantifies
219 how much they agree in practice.

220 **Connection to the EMNLP 2026 explainability theme.** Our work exemplifies a pragmatic form
221 of explainability: not a post-hoc rationalisation of an individual prediction, but a structural account of
222 *where* capability lives and thus *where* it can safely be updated. This reframes PEFT target selection
223 as an interpretability-grounded design choice rather than a convention inherited from the original
224 LoRA paper.

225 **Failure modes.** When task-relevant knowledge is stored in the forgetting-vulnerable region,
226 TG-LoRA under-performs standard LoRA on the *current* task. We observe this for **[PEND-**
227 **ING: failure_mode_tasks]** (see §5.4). Practitioners deploying TG-LoRA for a task whose knowledge
228 is known to reside in late MLPs (e.g., narrow factual domains) should raise τ to include more vulner-
229 able components, trading a small forgetting regression for current-task accuracy.

230 **Positioning vs. LOFIT and concurrent component-level work.** LOFIT [Yin et al., 2024] also
231 localises LoRA updates to a subset of components, but it picks them by *new-task adaptation leverage*.
232 TG-LoRA picks them by *prior-task resistance*. The two objectives point at overlapping but distinct
233 subsets (Jaccard = **[PENDING: lofit_tglora_jaccard]**); combining them is a natural follow-up.

234 Concurrent work [Anonymous, 2026, 2024] has begun to characterise forgetting at component
235 granularity, but via gradient magnitudes and per-layer learning-rate scaling respectively, not via PEFT
236 target selection.

237 7 Conclusion

238 We mapped the per-component forgetting topology of Qwen3-8B and Llama-3.1-8B via causal
239 tracing, introduced the transferable Forgetting Vulnerability Score (FVS), and showed that restricting
240 LoRA to the 50% most resistant components (TG-LoRA) reduces average prior-task forgetting by
241 **[PENDING: forget_reduction_pct]** while matching standard LoRA on downstream tasks. Beyond
242 the PEFT recipe, the topology yields a *mechanistic* account of forgetting: early-layer and anchor-
243 head resistance is consistent with a division between task-agnostic linguistic competence and task-
244 specific key-value storage in late-layer MLPs. This reframes forgetting as a question about *which*
245 *representational real-estate is being re-used*, and gives practitioners an interpretability-grounded rule
246 for choosing which modules to adapt and which to preserve.

247 Limitations

248 We restrict our study to two English-centric dense decoder-only models in the 8B class (Qwen3-8B
249 and Llama-3.1-8B) and to a single architectural family of attention (GQA). We do not test cross-
250 family transferability to mixture-of-experts (e.g., Llama 4 Scout, Mixtral) or to different parameter
251 scales; the claim that FVS transfers is therefore scoped to *dense 8B-class architectures sharing*
252 *GQA-style projection layouts*. We also restrict evaluation to English; multilingual forgetting may
253 exhibit different topology.

254 Our causal tracing uses *single-component* ablation and therefore under-measures effects that emerge
255 from multi-component interactions. Forgetting that is distributed across a small circuit (e.g., paired
256 induction heads, or an attention-head / MLP chain) is attributed only partially to each component in
257 isolation. Pairwise ablation is computationally quadratic in the number of components and is out of
258 scope for this submission; we discuss it as follow-up work.

259 We operate at projection granularity rather than attention-head granularity. LoRA’s implementation
260 targets full projections, so the TG-LoRA target set is naturally projection-level; extending both the
261 trace and the adapter to per-head granularity requires a custom LoRA wrapper and is left to future
262 work.

263 Within the 144-run main matrix ($2 \text{ models} \times 9 \text{ methods} \times 3 \text{ seeds} \times 4 \text{ stages}$) we do not perform an
264 exhaustive hyperparameter search for TG-LoRA’s threshold τ ; we report a three-point sensitivity
265 sweep (§5.4) rather than an optimal- τ curve.

266 Finally, the reversed-order ablation tests only one alternative task permutation, not the full $4! = 24$
267 permutation set. We report whether FVS is stable under the single most-informative ordering swap
268 (fully reversed) rather than across all orderings.

269 Broader Impacts

270 **Data & models.** This work studies fine-tuning dynamics on publicly available models (Qwen3-8B
271 under Apache 2.0; Llama-3.1-8B under the Llama-3 Community License) and public benchmarks
272 (CoNLL-2003, SQuAD-v1.1, XSum, CodeAlpaca, HumanEval). We do not release new data and we
273 do not train on scraped web content. All model training respects the licence terms of the underlying
274 checkpoints; we do not evaluate on held-out contamination of the benchmarks.

275 **Intended uses.** TG-LoRA is intended to reduce the compute and capability cost of adapting LLMs
276 while preserving prior capabilities, including safety-relevant behaviours learned during alignment.
277 The forgetting-topology view gives practitioners a principled tool to *avoid* the unintended erasure of
278 safety properties during downstream fine-tuning, which is an otherwise common failure mode.

279 **Potential misuse.** A methodology that identifies which components carry prior-task behaviour
280 could, in principle, be inverted to identify which components carry *safety* behaviour and to then adapt

or overwrite them specifically — the “refusal jailbreak via targeted editing” variant of this concern has been demonstrated in prior ROME-style work. We do not provide recipes or datasets that make this attack easier. All components of our pipeline assume a single known task sequence and measure forgetting against held-out benchmarks, not safety classifiers.

Environmental cost. Total compute for this paper is estimated in Appendix A using the emissions factor of the grid region of our compute provider. We report this explicitly rather than amortising across groups.

Human annotation. No new human annotation was collected. All labels come from the original benchmark releases. No study participants were involved.

References

- Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *ECCV*, 2018.
- Anonymous. Intelligent learning rate distribution to reduce catastrophic forgetting in transformers. *arXiv:2404.01317*, 2024.
- Anonymous. Mixture of LoRA experts for continual information extraction. In *Findings of EMNLP*, 2025a.
- Anonymous. Spurious forgetting in continual learning of language models. In *ICLR*, 2025b.
- Anonymous. Mechanistic analysis of catastrophic forgetting in large language models during continual fine-tuning. *arXiv:2601.18699*, 2026. Uses gradient attribution; reports 15–23% of heads severely disrupted, lower layers more susceptible.
- Sahil Chaudhary. Code Alpaca: An instruction-following Llama model for code generation. <https://github.com/sahil280114/codealpaca>, 2023.
- Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *EMNLP*, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- James Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. VeRA: Vector-based random matrix adaptation. In *ICLR*, 2024.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *ICML*, 2024.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

- 323 Yun Luo, Zhicheng Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study
324 of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint*
325 *arXiv:2308.08747*, 2023.
- 326 Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The
327 sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- 328 Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual
329 associations in GPT. In *NeurIPS*, 2022.
- 330 Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass editing
331 memory in a transformer. In *ICLR*, 2023.
- 332 Meta AI. The Llama 3 herd of models. *arXiv:2407.21783*, 2024.
- 333 Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don’t give me the details, just the summary!
334 Topic-aware convolutional neural networks for extreme summarization. In *EMNLP*, 2018.
- 335 Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads.
336 *Transformer Circuits Thread*, 2022.
- 337 Qwen Team. Qwen3 technical report. *arXiv:2505.09388*, 2025.
- 338 Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions
339 for machine comprehension of text. In *EMNLP*, 2016.
- 340 Lee Sharkey et al. Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.
- 341 Zhaorui Shi et al. Self-distillation bridges distribution gap in language model fine-tuning. In *ACL*,
342 2024.
- 343 Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task:
344 Language-independent named entity recognition. In *CoNLL*, 2003.
- 345 Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and
346 Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In
347 *NeurIPS*, 2020.
- 348 Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. In-
349 terpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *ICLR*,
350 2023.
- 351 Liyuan Wang et al. Continual learning of large language models: A comprehensive survey. *ACM*
352 *Computing Surveys*, 2025.
- 353 Fangcong Yin, Xi Ye, and Greg Durrett. LOFIT: Localized fine-tuning on LLM representations. In
354 *NeurIPS*, 2024.
- 355 Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and
356 Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *ICLR*,
357 2023.

358 A Reproducibility

359 **Code and data.** All code, configurations, and per-cell result JSON files are provided as anonymised
360 supplementary material for peer review, and will be released under the MIT licence at a permanent
361 URL upon acceptance. Datasets are publicly distributed under their original licences (CoNLL-
362 2003, SQuAD-v1.1, XSum, CodeAlpaca-20k, HumanEval). The CoNLL-2003 source used is the
363 parquet mirror tomaarsen/conll2003 on the Hugging Face Hub, which has schema identical to
364 the canonical conll2003 release.

365 **Hardware.** NVIDIA A40 48GB GPUs on a commercial GPU cloud. bf16 mixed precision, gradient
366 checkpointing enabled.

Software. Python 3.11, PyTorch 2.4.1 + CUDA 12.4, transformers $\geq$ 5.5.0, peft $\geq$ 0.19.0, trl $\geq$ 1.1.0, datasets $\geq$ 4.0.0, accelerate $\geq$ 1.1.1. Exact pinned versions in code/requirements.txt in the supplement.

Hyperparameters. See Table 2.

Hyperparameter	Value
Optimiser	AdamW
β_1, β_2	0.9, 0.999
Weight decay	0.01
Learning rate (LoRA methods)	2×10^{-4}
Learning rate (full FT)	2×10^{-5}
Warmup ratio	0.03
Effective batch size	16 (4 per-device $\times$ 4 accum)
Epochs per stage	3
Max sequence length	1024 (2048 for XSum)
LoRA r	16
LoRA α	32
LoRA dropout	0.05
EWC λ	500
SDFT α (mix weight)	0.5
Replay fraction (LoRA+Replay)	0.10
TG-LoRA τ	median FVS
Seeds	{42, 1337, 2024}

Table 2: Training hyperparameters, shared across all methods unless specified. Learning rate was selected on a held-out slice of stage-0 data by validation forgetting.

370

371 **Seeds.** {42, 1337, 2024}. All tables report mean $\pm$ standard deviation over seeds.

372 **Statistical tests.** Paired t -test across seeds for forgetting reductions; Wilcoxon signed-rank as a
373 non-parametric fallback. Spearman’s ρ for FVS transferability; Pearson r reported in the appendix
374 for sensitivity. Significance threshold $p < 0.05$ unless stated.

375 **Compute and carbon.** Total compute: [PENDING: total_a40_hrs] A40-hours. Carbon estimate:
376 kgCO₂e = A40-hrs $\times$ 0.300 kW $\times$ grid intensity, using the emissions factor of the specific grid
377 region of the compute provider at run-time as reported by the provider’s carbon dashboard. Estimated
378 total: [PENDING: carbon_kgco2e] kgCO₂e.

379 **Assets and licenses.** See Table 3 for every external asset used, its source, and its licence. New
assets introduced by this paper (the released code) are licensed under Apache-2.0.

Asset	Source / Hub ID	Licence
Qwen3-8B (base)	Qwen/Qwen3-8B	Apache-2.0
Llama-3.1-8B (base)	meta-llama/Llama-3.1-8B	Llama-3.1 Community
CoNLL-2003 (NER)	tomaarsen/conll2003	CC-BY-4.0
SQuAD (QA)	rajpurkar/squad	CC-BY-SA-4.0
XSum (Summarisation)	EdinburghNLP/xsum	Research-use (BBC)
HumanEval (Code)	openai/humaneval	MIT
transformers/peft/trl/datasets	HuggingFace	Apache-2.0
PyTorch	pytorch.org	BSD-3-Clause
This paper’s code release	anonymous mirror at submission	Apache-2.0

Table 3: External assets consumed by the experiments and their licences. We do not redistribute any of the upstream datasets or model weights; loading is delegated to the HuggingFace Hub at the pinned revisions recorded in code/requirements.txt. We release only analysis code.

380

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state three claims: (i) per-component forgetting follows a structural *topology* that is causally traceable, (ii) the resulting Forgetting Vulnerability Score (FVS) transfers across two open-weight 8B models within a measurable Spearman bound, and (iii) restricting TG-LoRA to the bottom-50% FVS components reduces average prior-task forgetting on a four-stage sequence relative to standard LoRA at matched parameter count. Each claim is mapped to a results subsection: (i) Section 5.1, (ii) Section 5.2, (iii) Section 5.3. Numerical magnitudes for each claim are reported with three seeds and paired-*t* significance tests.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations section enumerates: (i) two model families (Qwen3-8B, Llama-3.1-8B), so cross-family transfer beyond dense decoder transformers is untested; (ii) causal tracing operates at projection granularity rather than head-level (head-level forgetting is explicit future work); (iii) the four-task sequence is fixed in scope (NER $\rightarrow$ QA $\rightarrow$ Sum $\rightarrow$ Code) and a reverse-order ablation is reported but not a full Latin-square; (iv) three seeds per cell, no learning-rate or batch-size hyperparameter sweep; (v) supervised fine-tuning only; no RLHF/DPO; (vi) MoE architectures are excluded because expert-routing complicates causal tracing; (vii) FVS is computed at the component-mean level, not at the per-example level.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is purely empirical. The FVS definition (Section 3) is a normalised aggregate of measured task-pair forgetting deltas; no formal theorem or proof is asserted.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 fully specifies the four-task sequence (CoNLL-2003 NER, SQuAD QA, XSum Summarisation, HumanEval Code), the two base models with HuggingFace revision pins (Qwen/Qwen3-8B, meta-llama/Llama-3.1-8B), the LoRA configuration ($r=16$, $\alpha=32$, target modules $\{q, k, v, o, \text{up, down, gate}\}$), the optimizer (AdamW, $\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01, cosine schedule, 5% warmup), the per-task epoch counts, the sequence length (2048), and the precision (bf16). Full hyperparameters are tabulated in Appendix A, Table 2. The matrix orchestrator (code/scripts/run_matrix.py) is committed and resumable.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All four downstream datasets are publicly available on the HuggingFace Hub (tomaarsen/conll2003 parquet mirror, rajpurkar/squad, EdinburghNLP/xsum, openai/humanEval); we do not redistribute them. Code is released under an

Apache-2.0 license at an anonymous GitHub mirror linked from the supplementary zip during review and replaced with the canonical repository URL on acceptance. The release contains the orchestrator, causal-tracing implementation, EWC/SDFT baselines, the HumanEval pass@1 sandbox, plotting scripts, and a one-command reproduction recipe (python code/scripts/run_matrix.py -config experiments/configs/qwen3_8b_main.yaml -resume).

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4, “Setup,” and Appendix A together specify train/validation/test splits for each of the four tasks, the optimizer (AdamW), learning-rate schedule (cosine, 5% warmup), per-stage epoch counts, effective batch size, sequence length, precision (bf16), gradient accumulation, the full LoRA target-module set, and the causal-tracing intervention protocol (Gaussian-noise corruption at the residual stream of the source token, projection-level patch). Hyperparameters were chosen by single-seed validation on the smaller pilot model (Qwen3-1.7B) and held fixed across all 8B runs to avoid double-dipping; this protocol is stated in Section 4.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Each cell is run with three seeds ($\{0, 1, 2\}$). Tables report seed mean and standard deviation. The headline TG-LoRA-vs-LoRA forgetting comparison uses a paired t -test on per-seed average forgetting (paired by seed) and reports a p -value of [PENDING: tglora_vs_lora_p]. Cross-family FVS transfer is reported as Spearman’s ρ over architecturally-shared projection positions with rank-permutation p -values (Section 5.2). When comparing TG-LoRA against four additional baselines (DoRA, VeRA, LOFIT, random-target control), per-pair effect sizes and unadjusted p -values are reported in Appendix A; we do not claim significance against these four absent multiplicity correction.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All experiments run on a single NVIDIA A40 (48 GB) per cell at \$0.44/GPU-hour (RunPod SECURE cloud), with bf16 precision and gradient checkpointing. The full main matrix (six methods $\times$ three seeds $\times$ two models, 36 cells) consumes [PENDING: total_a40_hrs] A40-hours; the four reviewer-added baseline cells (DoRA, VeRA, LOFIT, random-target on Qwen3-8B with one seed) add an additional 32 A40-hours. Total dollar cost (including preempted-and-resumed pods, so no failed-run compute is hidden) is reported in Appendix A. Per-cell wall-clock breakdowns by stage (NER, QA, Sum, Code) are provided in the same appendix.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The research conforms with the NeurIPS Code of Ethics. No human subjects are involved at any stage. All four downstream benchmarks are publicly released under their original research-use licenses; we do not redistribute their content. The two base models (Qwen3-8B, Llama-3.1-8B) are open-weight and used under their respective licences. Trained adapter checkpoints from our matrix are not published (they encode no novel pre-training data and reproduce trivially from the released code).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader Impacts section discusses the positive impact (efficient continual fine-tuning that reduces compute waste and carbon cost compared to retraining from scratch when downstream tasks evolve, and a diagnostic tool, FVS, that practitioners can apply to predict which components will be eroded under sequential adaptation) and the potential negative impact (the same topology, used adversarially, could indicate which components an attacker should aim to corrupt to maximally degrade prior task performance — a concern shared with any localisation-based interpretability work). Mitigations: code is released, the method is published openly so detection benefits accrue symmetrically, and we do not release attack tooling.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases analysis code only. No new pre-trained model weights, image generators, or scraped datasets are introduced. The trained TG-LoRA adapters can be regenerated from the released orchestrator and are not redistributed.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Each used asset is credited and its licence is listed in Appendix A, Table 3: Qwen/Qwen3-8B (Apache-2.0), meta-llama/Llama-3.1-8B (Llama 3.1 Community License), tomaarsen/conll12003 (CC-BY-4.0; original CoNLL-2003 license preserved), rajpurkar/squad (CC-BY-SA-4.0), EdinburghNLP/xsum (research-use, original BBC content via news redistribution), openai/humanEval (MIT), HuggingFace transformers/peft/trl/datasets (Apache-2.0), PyTorch (BSD-3-Clause).

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is the source code release, which contains a top-level README.md, an installation recipe (requirements.txt), per-module docstrings for the seven core modules (forgetting_map/, evaluation/, scripts/), example invocations for each main entry point, and the FVS JSON schema documented in experiments/results/SCHEMA.md. The release is licensed Apache-2.0.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subjects research is involved at any stage.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

539 Answer: [N/A]
540 Justification: No human subjects are involved at any stage; IRB review is not applicable.
541
542 16. **Declaration of LLM usage**
543 Question: Does the paper describe the usage of LLMs if it is an important, original, or
544 non-standard component of the core methods in this research?
545 Answer: [N/A]
546 Justification: LLMs are the *subject* of study, not a methodological component. The causal
547 tracing, FVS aggregation, EWC/SDFT/replay baselines, TG-LoRA target selection, and
548 pass@1 evaluation pipelines are deterministic numerical procedures implemented in Py-
549 Torch, HuggingFace transformers, and peft. No LLM is used as part of the core method-
550 ology, scientific reasoning, or analysis. LLM-based assistance was used for manuscript
551 drafting and code review; per the NeurIPS LLM policy, this kind of writing assistance does
not require declaration as a methodological component.